



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Functions

CMSC 124 Lab 5

Prepared by: Rene Andre Bedonia Jocsing

Published: November 04, 2025

Welcome to the last leg of your language hacking journey. You've built a scanner, parser, and evaluator, then added scoping to your interpreter. Now it's time to add the features that will transform your interpreter from a sophisticated calculator into a real programming language: **control flow** and **functions**.

Control flow lets your programs make decisions and repeat operations. Functions let you package reusable chunks of code. Together, these features give your language the power to solve real problems, taking one step closer to Turing-completeness. A programming language without these constructs is like a car without a steering wheel—it can move, but you can't really go anywhere interesting.

This final lab represents the culmination of everything you've learned. By the end, you'll have implemented a complete, Turing-complete programming language. That's no small feat!

If-Else

Every useful program needs to make decisions. Should we execute this code or that code? Should we repeat this operation or move on? This is what control flow constructs enable.

The most fundamental control flow construct is the **if statement**. It evaluates a condition and executes different code depending on whether the condition is true or false:

```
1  if (temperature > 30) {  
2      print "It's hot!";  
3  } else {  
4      print "It's comfortable.";  
5  }
```

The condition (the expression in parentheses) gets evaluated to a boolean value using your truthiness rules from Lab 3. If it's truthy, execute the first branch. If it's falsey, execute the else branch (if one exists).

Your grammar might look like:

```
1  statement      → exprStmt | printStmt | varDecl | block | ifStmt ;  
2  ifStmt         → "if" "(" expression ")" statement ( "else" statement )? ;
```

Notice that the branches are statements, not just blocks. This allows single-statement branches without braces, though many style guides discourage this. It also enables **cascading if-else chains**. Like so:

```

1  if (score >= 90) {
2      grade = "A";
3  } else if (score >= 80) {
4      grade = "B";
5  } else if (score >= 70) {
6      grade = "C";
7  } else {
8      grade = "F";
9  }

```

This works because the `else if` is really an `else` containing another `if` statement.

The Dangling Else Problem

If you allow `if` statements without braces, you'll encounter a classic parsing ambiguity. Consider:

```

1  if (first)
2      if (second)
3          print "both";
4      else
5          print "???";

```

Which `if` does the `else` belong to? The grammar is ambiguous—it allows two different parse trees. Most languages resolve this by having the `else` bind to the nearest `if`. Your parser should naturally do this if you implement the grammar correctly, but be aware of the issue.

Loops

The second pillar of control flow is repetition. While you could achieve repetition with recursion and `if` statements (and some languages do!), loops are more intuitive for most programmers.

The **while loop** is the simplest. It repeatedly executes a statement as long as a condition remains true:

```

1  var i = 0;
2  while (i < 10) {
3      print i;
4      i = i + 1;
5  }

```

Your grammar may add:

```

1  statement      → ... | whileStmt ;
2  whileStmt      → "while" "(" expression ")" statement ;

```

The implementation is straightforward: evaluate the condition, and if truthy, execute the body statement. Then loop back and check the condition again. Repeat until the condition becomes falsey.

Most languages also include a **for loop**. The `for` loop is really just syntactic sugar—a more convenient way to write a common pattern. The classic C-style `for` loop has three parts: an initializer, a condition, and an increment:

```

1  for (var i = 0; i < 10; i = i + 1) {
2      print i;
3  }

```

This is equivalent to:

```

1  {
2      var i = 0;
3      while (i < 10) {
4          print i;
5          i = i + 1;
6      }
7  }

```

Your grammar might be:

```

1  statement      → ... | forStmt ;
2  forStmt        → "for" "(" ( varDecl | exprStmt | ";" )
3                  expression? ";"
4                  expression? ")" statement ;

```

The three parts are all optional, which is why you see the ? markers. An infinite loop would be `for(;;) { ... }`.

When executing a for loop, you can desugar it into the while loop equivalent during parsing or evaluation. This keeps your implementation simpler—you don't need separate logic for for loops.

Logical Operators

With control flow, you'll often need to combine conditions. **Logical operators** `and` and `or` (or `&&` and `||` in C-style syntax) fill this role:

```

1  if (temperature > 30 and humidity > 80) {
2      print "It's hot AND humid!";
3  }

```

These operators are special because they **short-circuit**. The `and` operator only evaluates its right operand if the left operand is true. The `or` operator only evaluates its right operand if the left operand is false. This isn't just an optimization—it affects program semantics.

Consider:

```

1  if (denominator != 0 and numerator / denominator > 1) {
2      print "Ratio is greater than 1";
3  }

```

If `denominator` is zero, the right side never evaluates, avoiding a division-by-zero error. Without short-circuiting, this code would crash.

Your grammar needs to place these operators at the right precedence level. For example:

```

1 expression    → assignment ;
2 assignment    → IDENTIFIER "=" assignment | logicOr ;
3 logicOr       → logicAnd ( "or" logicAnd )* ;
4 logicAnd      → equality ( "and" equality )* ;

```

The `or` operator has lower precedence than `and`, matching how we think about them in natural language.

Functions

Functions are one of the most important abstractions in programming. They let you:

- Package code for reuse
- Give meaningful names to operations
- Hide implementation details
- Reduce code duplication
- Build programs compositionally

A function declaration might look like:

```

1 fun greet(name) {
2   print "Hello, " + name + "!";
3 }

```

And you call it like:

```

1 greet("Alice");
2 greet("Bob");

```

Your grammar may extend to include function declarations and calls:

```

1 declaration    → funDecl | varDecl | statement ;
2 funDecl       → "fun" IDENTIFIER "(" parameters? ")" block ;
3 parameters     → IDENTIFIER ( "," IDENTIFIER )* ;
4
5 primary       → ... | IDENTIFIER | "(" expression ")" ;
6 call          → primary ( "(" arguments? ")" )* ;
7 arguments     → expression ( "," expression )* ;

```

Notice that calls are postfix operators on expressions. You can call any expression that evaluates to a callable value: `getFunction()`.

Function Objects and Closures

When you declare a function, what actually happens? You need to store information about the function so you can execute it later when it's called. Most implementations create a **function object** that contains:

- The function's parameter names
- The function body (as an AST node)
- The environment where the function was declared (for closures)

This function object gets stored in your environment just like any other value. The function's name is a variable that holds a reference to the function object.

The environment capture is crucial for **closures**—functions that “close over” and remember variables from their enclosing scope. Do closures exist in your language? If so, this is where you implement them.

```

1  fun makeCounter() {
2      var count = 0;
3      fun increment() {
4          count = count + 1;
5          return count;
6      }
7      return increment;
8  }
9
10 var counter = makeCounter();
11 print counter(); // 1
12 print counter(); // 2
13 print counter(); // 3

```

The `increment` function remembers the `count` variable even after `makeCounter` has returned. This is only possible if the function stores a reference to the environment where it was created.

Calling Functions

When a function is called, you need to:

1. Evaluate all argument expressions
2. Create a new environment for the function execution
3. The new environment's parent should be the function's closure environment (where it was declared), not the current environment (where it's being called)
4. Bind parameter names to argument values in the new environment
5. Execute the function body in this new environment
6. Return the result (if any)

This process is called **dynamic dispatch**—deciding at runtime which code to execute based on what function object you're calling.

Return Statements

Functions need a way to send values back to their callers. The `return` statement does this:

```

1  fun add(a, b) {
2      return a + b;
3  }
4
5  var result = add(3, 5);
6  print result; // 8

```

Your grammar adds:

```

1  statement      → ... | returnStmt ;
2  returnStmt     → "return" expression? ";" ;

```

The expression is optional—`return;` with no value is valid. This implicitly returns `nil` (or your language's equivalent).

Implementing `return` is tricky because it needs to unwind the call stack, potentially exiting through multiple nested statements and blocks. Many implementations use exceptions for control flow here. When you execute a `return` statement, you can trivially throw a special “return exception” that carries the return value. The function call catches this exception and uses the value as the function's result.

Native Functions

Before users can write functions, your interpreter needs at least one function they can call—otherwise there's no way to get the ball rolling. Most languages provide **native functions** (also called built-in or primitive functions) implemented in the host language.

For example, you might want a `clock()` function that returns the current time, or a `readLine()` function for user input. These can't be implemented in your language because they need to access host language capabilities.

Implement native functions as callable objects that execute host language code instead of interpreting AST nodes. Add them to your global environment before executing any user code.

Arity Checking

Functions have a specific number of parameters (their **arity**). Most languages require that calls provide exactly the right number of arguments:

```
1 fun add(a, b) {
2     return a + b;
3 }
4
5 add(1, 2);      // OK
6 add(1);         // Error: Expected 2 arguments but got 1
7 add(1, 2, 3);   // Error: Expected 2 arguments but got 3
```

Check arity when calling functions and report clear errors for mismatches. This catches bugs early.

Some languages allow default parameters or variadic functions (accepting any number of arguments). These are optional enhancements if you want to implement them.

Recursion

One beautiful consequence of your implementation is that recursion should “just work.” Since function names are variables in the environment, and that environment is accessible during function execution, a function can call itself:

```
1 fun fibonacci(n) {
2     if (n <= 1) return n;
3     return fibonacci(n - 1) + fibonacci(n - 2);
4 }
5
6 print fibonacci(10); // 55
```

Make sure your implementation handles this correctly. Test with recursive functions, including mutually recursive functions (where function A calls function B, which calls function A).

Laboratory Deliverables

Grammar Extension

Update your language's grammar to include control flow and functions. Add production rules for:

- If statements (with optional else)
- While loops
- For loops
- Logical operators (and, or)
- Function declarations

- Function calls
- Return statements

Document these in your README.md under the Grammar section. Your complete grammar should now describe a Turing-complete programming language.

Control Flow Implementation

Extend your parser and evaluator to support:

- If-else statements with correct precedence
- While loops
- For loops (can be desugared to while loops)
- Short-circuiting logical operators
- Proper scoping for loop variables

Function Implementation

Implement functions with:

- Function declaration statements
- Function call expressions
- Parameter binding
- Return statements (including early returns)
- Closures (capturing enclosing environment)
- Proper environment management for calls
- Arity checking with clear error messages

Native Functions

Add at least two native functions to your language. Suggested examples:

- `clock()`: Returns current time in seconds
- `print(value)`: Prints a value (if not already a statement)
- `readLine()`: Reads user input
- `toString(value)`: Converts value to string

Comprehensive Testing

Create test programs demonstrating:

- Conditional execution (if-else)
- Loops (while and for)
- Nested control flow
- Function declaration and calling
- Recursive functions
- Closures
- Higher-order functions (functions that take or return functions)
- Error handling (wrong arity, etc.)

You should have several script files showcasing your language's capabilities. These will be demonstrated during your final defense.

Language Showcase

Write at least one non-trivial program in your language that demonstrates its capabilities. Add this as an example program in your repository (perhaps in an `examples/` directory).

As always, commit your changes with meaningful messages. This is the final major addition to your interpreter—you should be proud of what you've built!

Expected Output

Control Flow:

```
1 > var x = 5;
2 > if (x > 3) print "x is big";
3 x is big
4 > if (x > 10) {
5 >     print "x is huge";
6 > } else {
7 >     print "x is not that big";
8 > }
9 x is not that big
10 > var i = 0;
11 > while (i < 3) {
12 >     print i;
13 >     i = i + 1;
14 > }
15 0
16 1
17 2
18 > for (var j = 0; j < 3; j = j + 1) {
19 >     print j;
20 > }
21 0
22 1
23 2
24 > if (true or false) print "yep";
25 yep
26 > if (false and true) print "nope"; else print "correct";
27 correct
```

Functions:

```
1 > fun greet(name) {
2 >     print "Hello, " + name + "!";
3 > }
4 > greet("Alice");
5 Hello, Alice!
6 > fun add(a, b) {
7 >     return a + b;
8 > }
9 > print add(3, 5);
10 8
11 > fun fibonacci(n) {
12 >     if (n <= 1) return n;
13 >     return fibonacci(n - 1) + fibonacci(n - 2);
14 > }
15 > print fibonacci(10);
16 55
17 > fun makeCounter() {
18 >     var count = 0;
19 >     fun increment() {
20 >         count = count + 1;
21 >         return count;
22 >     }
23 >     return increment;
24 > }
25 > var counter = makeCounter();
26 > print counter();
```



```

27  1
28  > print counter();
29  2
30  > print counter();
31  3
32  > greet();
33  [line 1] Runtime error: Expected 1 arguments but got 0.
34  > add(1, 2, 3);
35  [line 1] Runtime error: Expected 2 arguments but got 3.

```

Example script file (fibonacci.txt):

```

1  // Recursive Fibonacci implementation
2  fun fibonacci(n) {
3      if (n <= 1) {
4          return n;
5      }
6      return fibonacci(n - 1) + fibonacci(n - 2);
7  }
8
9  // Print first 15 Fibonacci numbers
10 for (var i = 0; i < 15; i = i + 1) {
11     print "fib(" + i + ") = " + fibonacci(i);
12 }

```

Output:

```

1  $ java -jar language.jar fibonacci.txt
2  fib(0) = 0
3  fib(1) = 1
4  fib(2) = 1
5  fib(3) = 2
6  fib(4) = 3
7  fib(5) = 5
8  fib(6) = 8
9  fib(7) = 13
10 fib(8) = 21
11 fib(9) = 34
12 fib(10) = 55
13 fib(11) = 89
14 fib(12) = 144
15 fib(13) = 233
16 fib(14) = 377

```

Example script file (closures.txt):

```

“kotlin // Demonstrate closures fun makePrinter(prefix) { fun print(message) { print prefix + “.” + mes-
sage; } return print; }

var errorPrinter = makePrinter(“ERROR”); var infoPrinter = makePrinter(“INFO”);

```